UNIVERSIDAD CATÓLICA DE CÓRDOBA
JESUITAS

Calidad de Software

Mag. Pablo Virgolini

21 de septiembre de 2026

¿Avanzamos?



¿Qué es automatizar el testing?

¿Qué es automatizar el testing?

Automatizar el testing es usar software para ejecutar pruebas, comparar el resultado obtenido con el esperado y reportar el resultado, sin intervención humana en cada corrida.

Prueba manual

Una persona ejecuta y observa

- Flexible: aprovecha la intuición y el ojo humano.
- Costo alto y constante en cada ejecución.
- Ideal para explorar, evaluar usabilidad y probar lo nuevo.

Prueba automatizada

Un script ejecuta y verifica

- Rápida y repetible: corre sin supervisión.
- Costo inicial alto, costo por ejecución bajo.
- Ideal para regresión y tareas repetitivas.

Conexión con QA vs. QC: ejecutar pruebas (manuales o automáticas) es actividad de QC; decidir qué, cuándo y cómo automatizar es una decisión de QA.

Prueba automatizada

Toda prueba automatizada, sin importar la herramienta, sigue la misma estructura (patrón AAA: Arrange, Act, Assert).

1. Preparar



Arrange

Datos de prueba, entorno y precondiciones: abrir la página, crear el usuario.

2. Ejecutar



Act

La acción sobre el sistema: completar un formulario, hacer clic, enviar un request.

3. Verificar



Assert

Comparar el resultado obtenido con el esperado mediante aserciones.

4. Reportar y limpiar



Registrar

Registrar resultado y evidencia; cerrar el navegador y restaurar el estado.

Sin aserciones no hay prueba. Un script que solo navega y no verifica nada es una demo, no un test.

¿Qué automatizar?

Pruebas de regresión

Verificar que lo que funcionaba sigue funcionando después de cada cambio.

Tareas repetitivas

Flujos que se ejecutan una y otra vez: login, alta, búsqueda, checkout.

Pruebas de humo (smoke)

Verificación rápida de que el build es estable antes de invertir más pruebas.

API e integración

Contratos, códigos de estado y datos entre servicios: rápidas y estables.

Combinaciones de datos

La misma prueba con muchos datos (data-driven): validaciones y límites.

Múltiples entornos

El mismo flujo en varios navegadores, dispositivos o versiones.

Regla práctica: automatizar lo que es estable, repetitivo, de alto riesgo y se ejecuta con frecuencia.

¿Qué no automatizar?

Usabilidad y experiencia de usuario (UX)

¿Es intuitivo? ¿Es agradable de usar? Requiere criterio, empatía y sensibilidad humana.

Pruebas exploratorias

Se basan en intuición y aprendizaje sobre la marcha: un guion fijo las anula.

Requisitos inestables

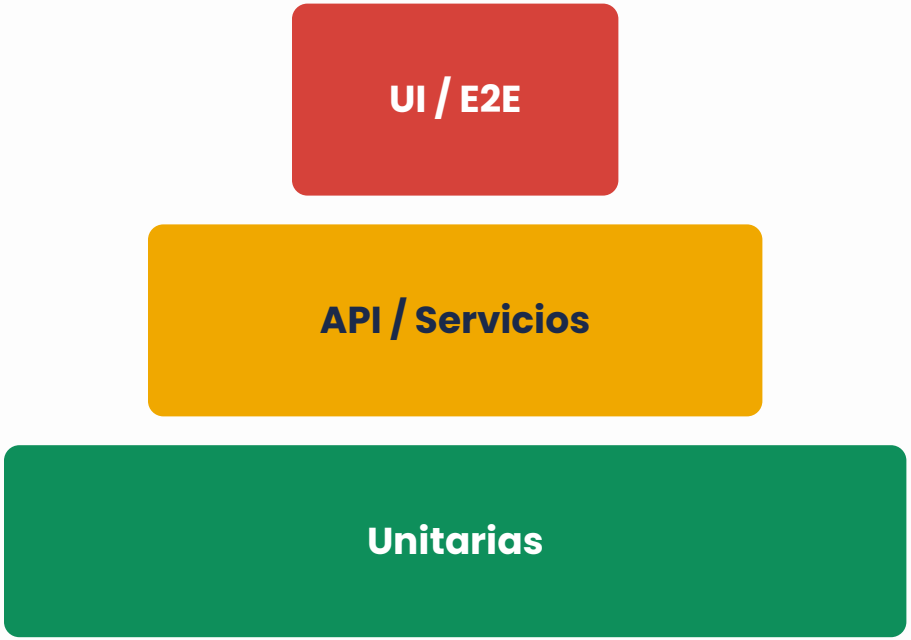
Si la funcionalidad cambia cada sprint, el script se rompe antes de amortizarse.

Pruebas de una sola vez

Escribir y mantener un script cuesta más que ejecutar la prueba a mano una vez.

Un script solo encuentra lo que alguien le enseñó a buscar. La exploración humana encuentra cosas nuevas.

La pirámide de la automatización



Pocas pruebas, lentas y mas sensibles

Simulan al usuario en el navegador.

Herramientas: Selenium · Cypress

Cantidad media, rápidas y estables

Verifican la lógica sin pasar por la interfaz.

Herramientas: Postman · Newman

Muchas pruebas, muy rápidas y baratas

Prueban funciones y clases de forma aislada.

Herramientas: pytest · JUnit

Cuanto más abajo, más rápido y barato el feedback: esta clase se concentra en las dos capas superiores (UI y API).

Trabajamos los conceptos: ¿Automatizar o no?

Clasifiquemos cada situación en grupo

1. Verificar el login con 20 combinaciones de usuario y contraseña.

2. Evaluar si el nuevo diseño del checkout resulta intuitivo.

3. Ejecutar 300 casos de regresión antes de cada release.

4. Descubrir fallas navegando libremente la app recién construida.

5. Validar que GET /users devuelve 200 y una lista no vacía.

6. Probar una funcionalidad que el cliente cambia todas las semanas.

7. Verificar el flujo de compra en Chrome, Firefox y Edge.

8. Probar una migración de datos que se hará una única vez.

Para cada una: ¿automatizar o dejarla en manos de una persona? Justifiquen con los criterios vistos.

Ventajas de la automatización

Cuando se elige bien qué automatizar, los beneficios se multiplican con cada ejecución.

Velocidad

Una suite de cientos de casos corre en minutos y en paralelo. Feedback casi inmediato.

Repetibilidad

Los mismos pasos, cada vez. Sin cansancio, sin distracciones, sin interpretaciones.

Precisión

Compara resultados con exactitud: no se le escapa un valor, un campo ni un código de estado.

Detección temprana

En CI/CD cada commit dispara las pruebas: el defecto aparece minutos después de introducirlo.

También: ejecución fuera de horario, más combinaciones de datos, evidencia y reportes automáticos, y más tiempo para pruebas exploratorias.

Detección temprana: pruebas en CI/CD

La integración continua ejecuta las pruebas automáticamente ante cada cambio; si algo falla, el pipeline se detiene.



Más rápidas y baratas primero → más lentas y costosas después

Todo en verde: el cambio avanza a la siguiente etapa.

Falla una etapa: el pipeline se corta y avisa a quien hizo el commit.

Cuanto antes se detecta un defecto, más barato es corregirlo.

Curva de aprendizaje y costo inicial

Automatizar es una inversión: el costo se concentra al principio y se amortiza con cada ejecución.

Curva de aprendizaje

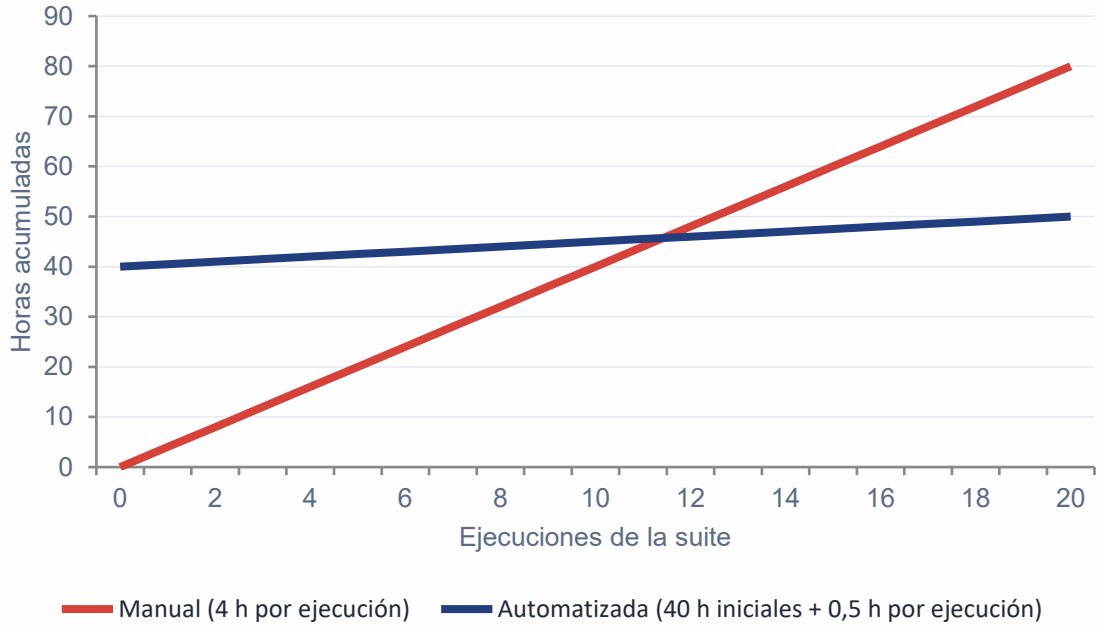
- Programación básica (variables, funciones, clases).
- Lenguaje y API del framework elegido.
- Sincronización: esperas explícitas vs. fijas.
- Ejecución en pipelines de CI/CD.

Costo inicial

- Tiempo para diseñar el framework y la estructura.
- Infraestructura: navegadores, runners y agentes.
- Capacitación del equipo y perfiles técnicos.
- Datos de prueba y entornos estables.
- Licencias, si la herramienta las requiere.

Automatizar primero unos pocos casos de alto valor (humo y regresión crítica) y crecer desde ahí.

Punto de equilibrio: ¿cuándo se paga?



≈ 12

ejecuciones para llegar al punto de equilibrio

Supuestos del ejemplo (ilustrativos)

- Suite manual: 4 h por ejecución.
- Automatizarla: 40 h de desarrollo.
- Mantenimiento: 0,5 h por ejecución.
- Equilibrio: 40 ÷ (4 − 0,5) ≈ 11,4.

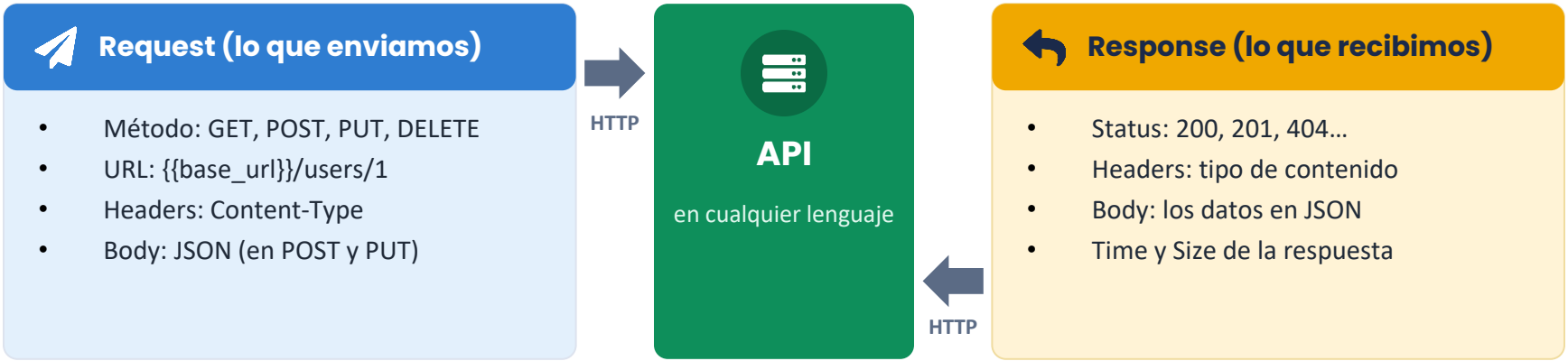
No todo es automatizable: si una prueba se ejecuta pocas veces, cambia seguido o exige criterio humano, nunca llega al punto de equilibrio.

Herramientas y frameworks

Postman (API)

¿Cómo funciona Postman?

Postman es un cliente HTTP: arma una request, la envía a la API, recibe la response y evalúa el resultado con aserciones.



Tests (JavaScript): pm.test y pm.expect comparan la response con lo esperado, y cada aserción termina en PASS o FAIL.

No ejecuta ni conoce el código de la API: solo se comunica por HTTP. Por eso prueba cualquier API como una caja negra.

Los bloques de Postman

Organización

Colección

GET	/health
POST	/users
POST	/login
DELETE	/users/{id}

Variables: {{base_url}}

Si existe en varios niveles gana el más específico: local › entorno › colección › global.

Ciclo de una request

1. **Pre-request script**
   Opcional: prepara datos o variables antes de enviar.
2. **Request**
   Se envía por HTTP a la API.
3. **Response**
   Llega el estado, los headers y el cuerpo.
4. **Tests**
   Las aserciones evalúan la response: PASS o FAIL.

Send ejecuta una request · Collection Runner ejecuta la colección completa en orden · Newman la ejecuta desde la terminal (CI/CD).

CASO PRACTICO

Postman + api_demo.py

Verificación con pruebas automáticas

Práctica: API local + Postman

Utilizamos una API propia: corre en nuestra computadora y la podemos probar las veces que sea necesario.



Objetivo: verificar con pruebas automáticas que la API cumple su contrato (códigos de estado y datos).

La API de práctica: endpoints

Método	Ruta	Qué hace	Respuestas
GET	`/health`	Verifica que la API está viva	200
GET	`/users`	Lista los usuarios (acepta ?rol=admin)	200
GET	`/users/{id}`	Devuelve un usuario	200 · 404
POST	`/users`	Crea un usuario (nombre y email obligatorios)	201 · 400
DELETE	`/users/{id}`	Elimina un usuario	204 · 404
POST	`/login`	Valida email y contraseña	200 · 401

Datos iniciales: 3 usuarios (Ana Pérez es admin). Viven en memoria: al detener la API se pierden.

Login válido: ana@ejemplo.com / clave123. Cualquier otra combinación devuelve 401.